6. Multimodal Visual Understanding + SLAM Navigation

Introduction

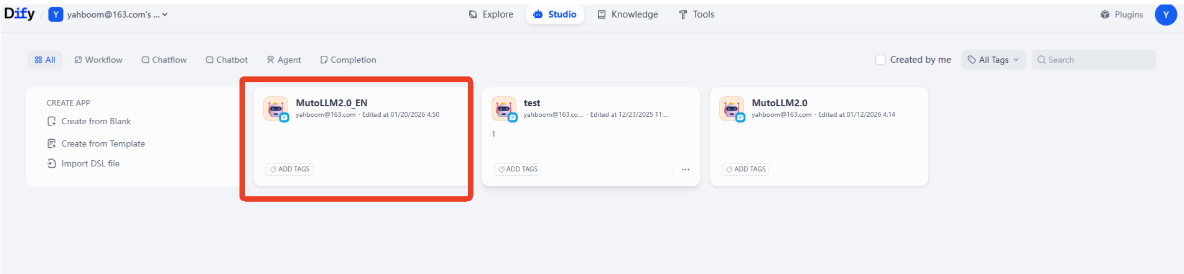
In this tutorial, we will learn how to issue commands via chat with Muto in the Dify interface.

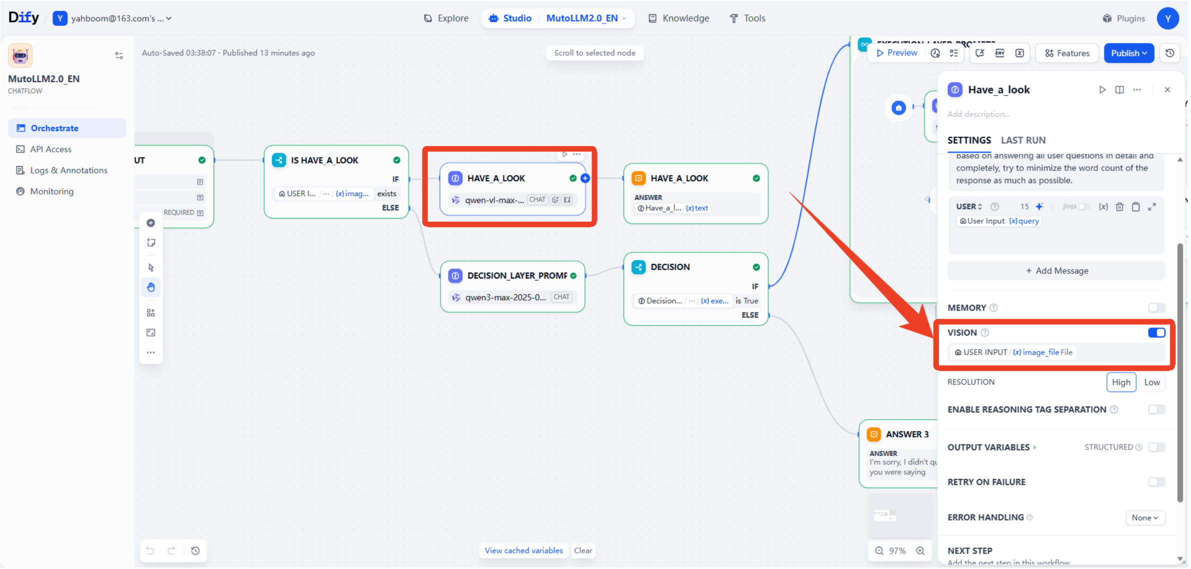
Operation Steps

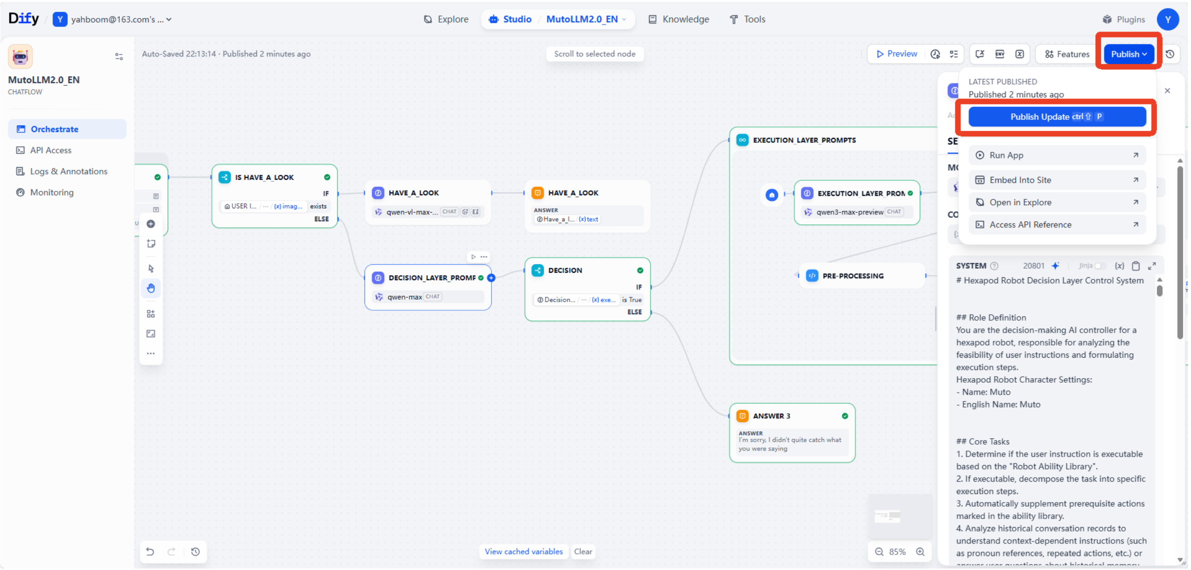
Preparatory Steps

This lesson requires that the Dify terminal be launched beforehand, and all configuration items have been configured as in the previous section.

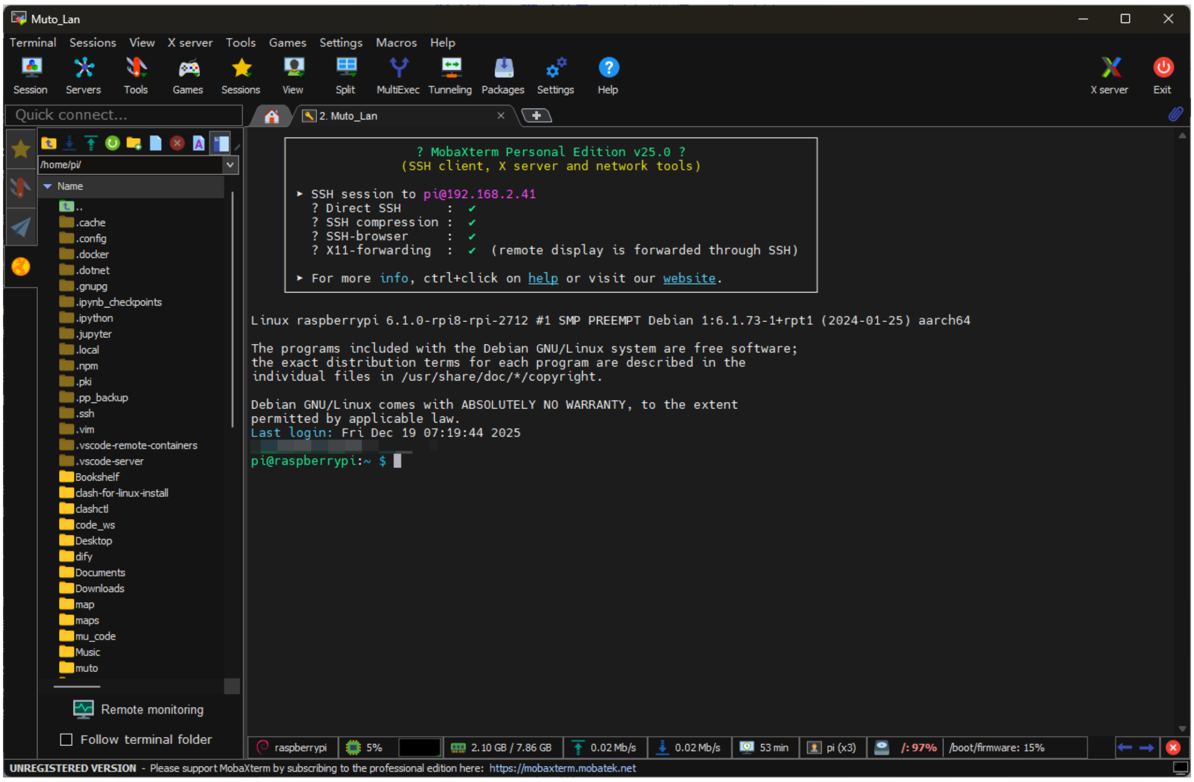
```
#!/usr/bin/env bash

# ==========================================
# User Configuration Area
# ==========================================

# 1. Voice Module Configuration
# ============================

# 1.1 Provider & Keys

export ALIBABA_API_KEY="sk-[illegible]20"

export FORCE_VOICE_PROVIDER="xunfei"

# 1.2 General Voice Settings
export VOICE_LANGUAGE="en"

# 2. Dify Configuration
# Auto-detect IP from wlan0
export DIFY_HOST=$(ip addr show wlan0 2>/dev/null | grep "inet " | awk '{print $2}' | cut -d/ -f1 | head -n 1)
# Fallback if wlan0 has no IP
if [ -z "$DIFY_HOST" ]; then
    export DIFY_HOST="127.0.0.1"
fi
```

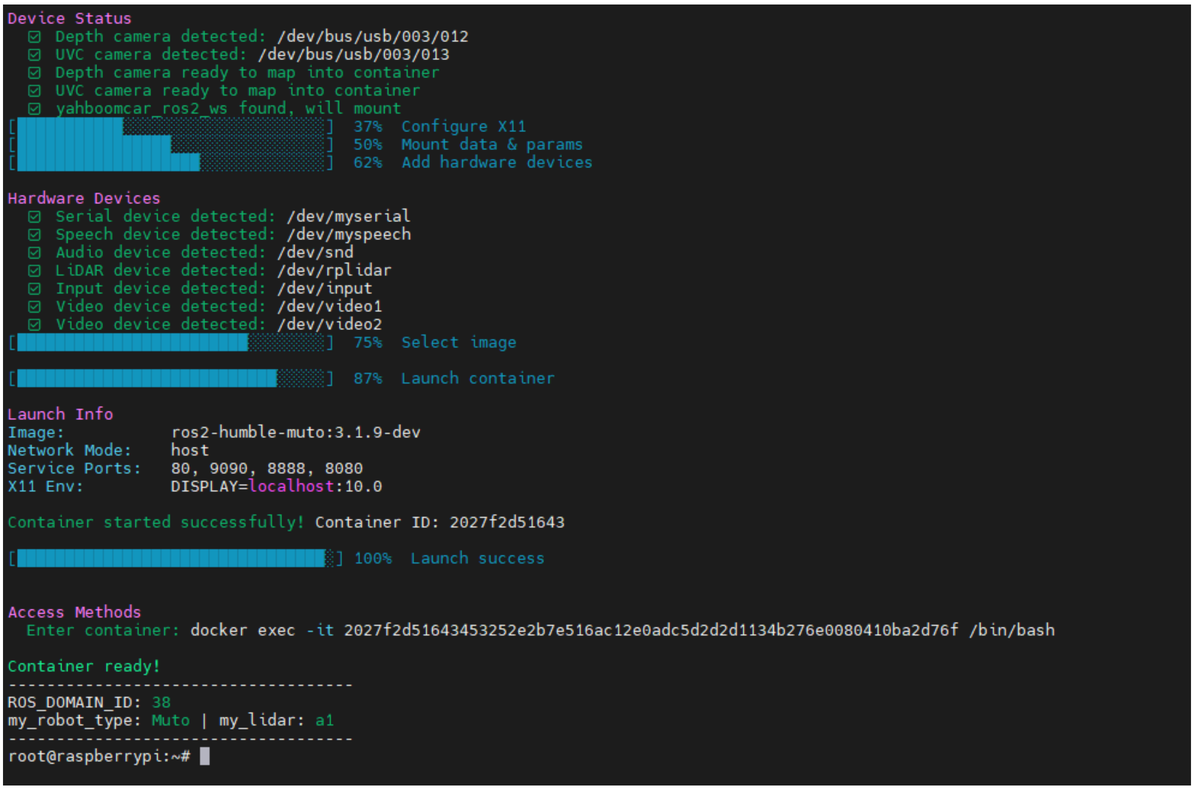






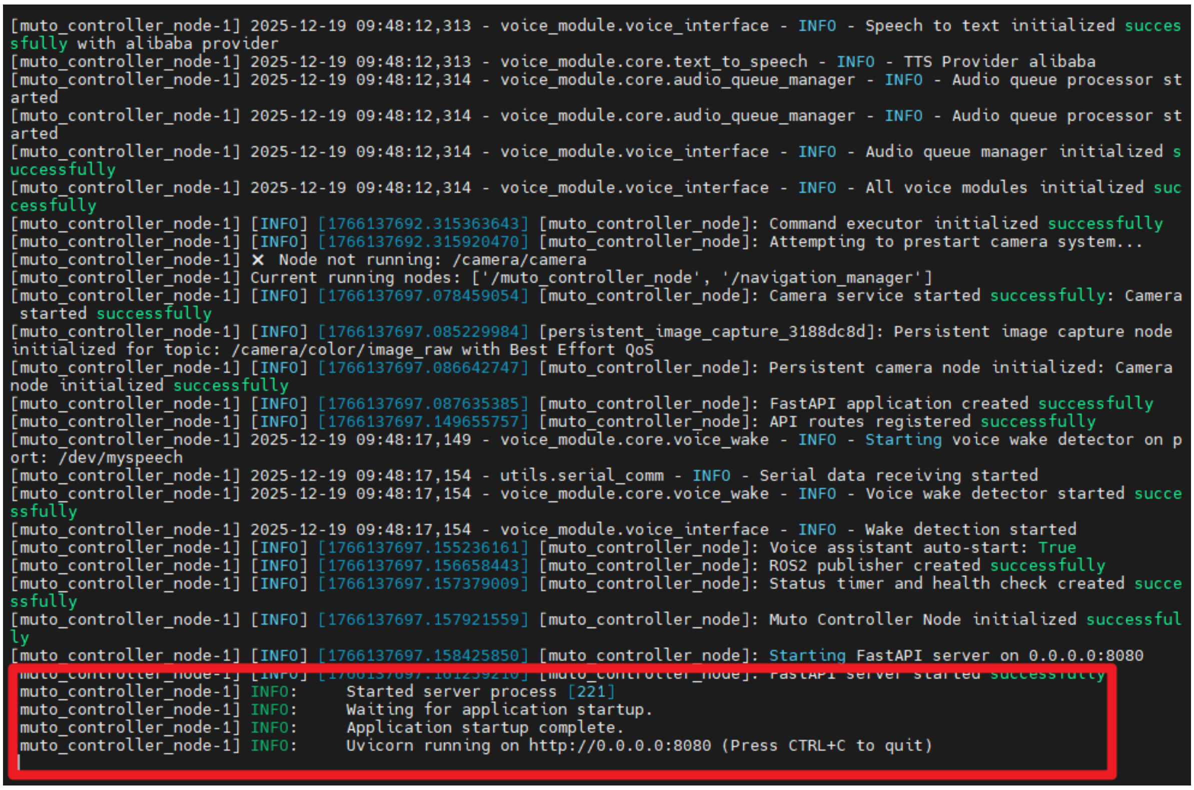
This lesson requires MutoRS to be powered on. Following the instructions in the previous chapters, start the large model terminal in Docker.





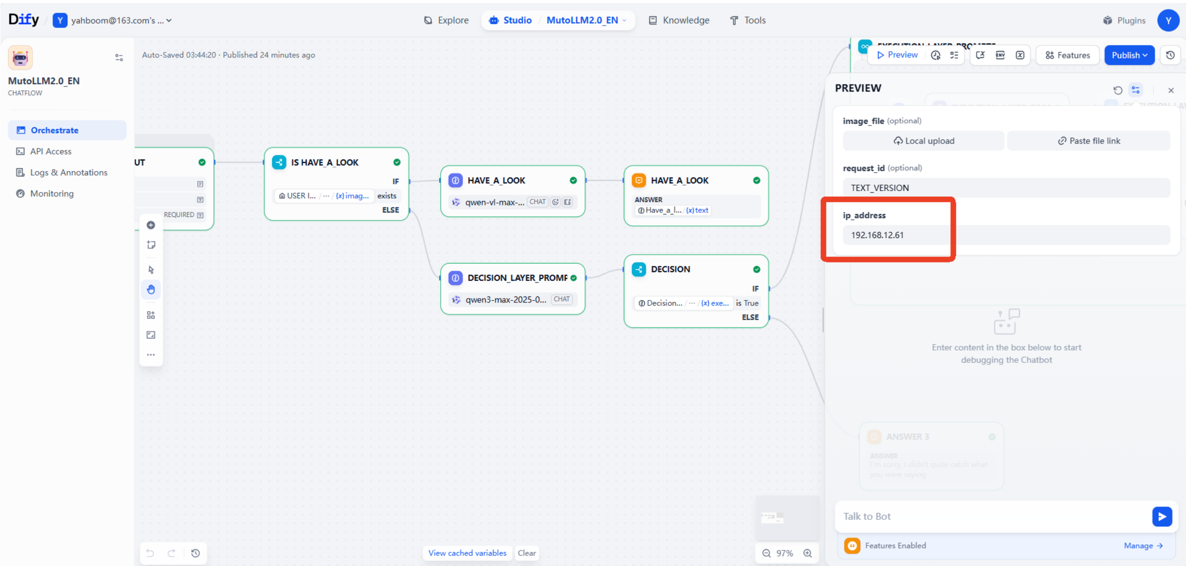
In this tutorial, we will use a plain text version of the large model, starting it with the `--no-voice` parameter. Using the `--no-voice` parameter eliminates the worry of being interrupted by external voice prompts during use!

```
cd muto-llm-2.0/ && ./rebuild_and_launch.sh --no-voice
```



Getting Started

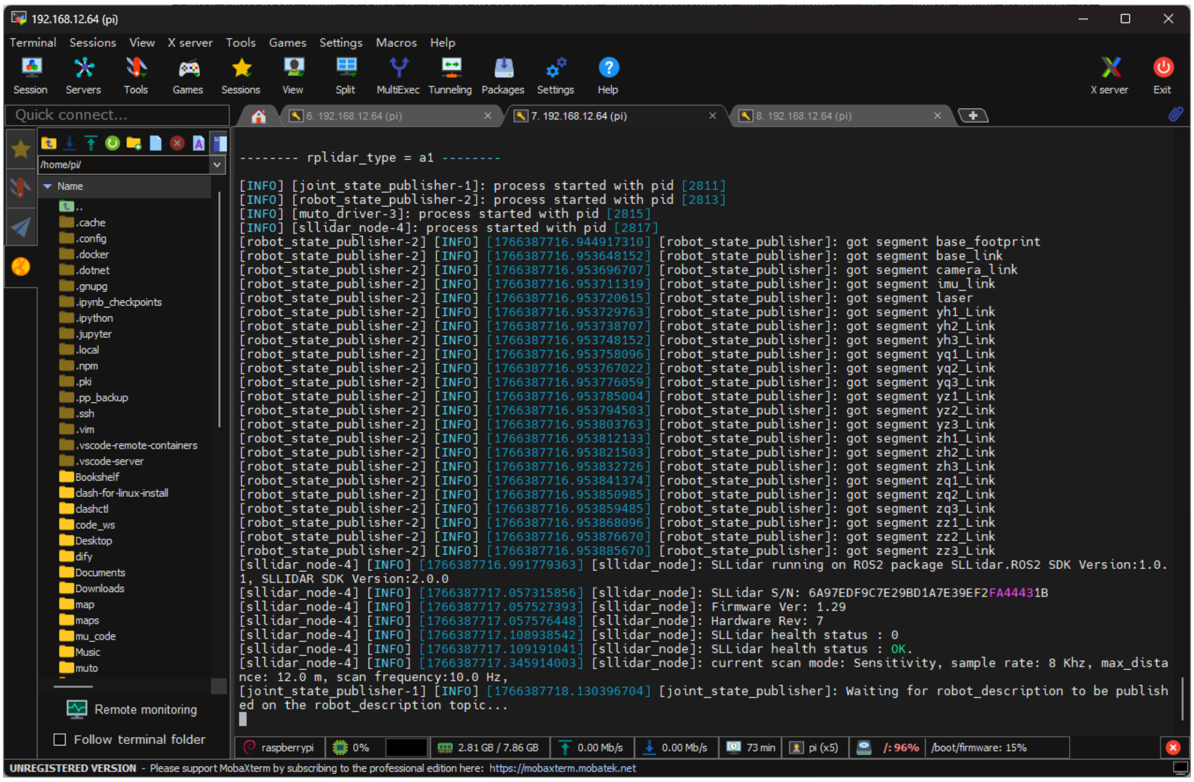
For the large text model, you need to enter the Muto IP address here, which is the address used to access the Dify page.



For the navigation task, we also need to open multiple terminals.

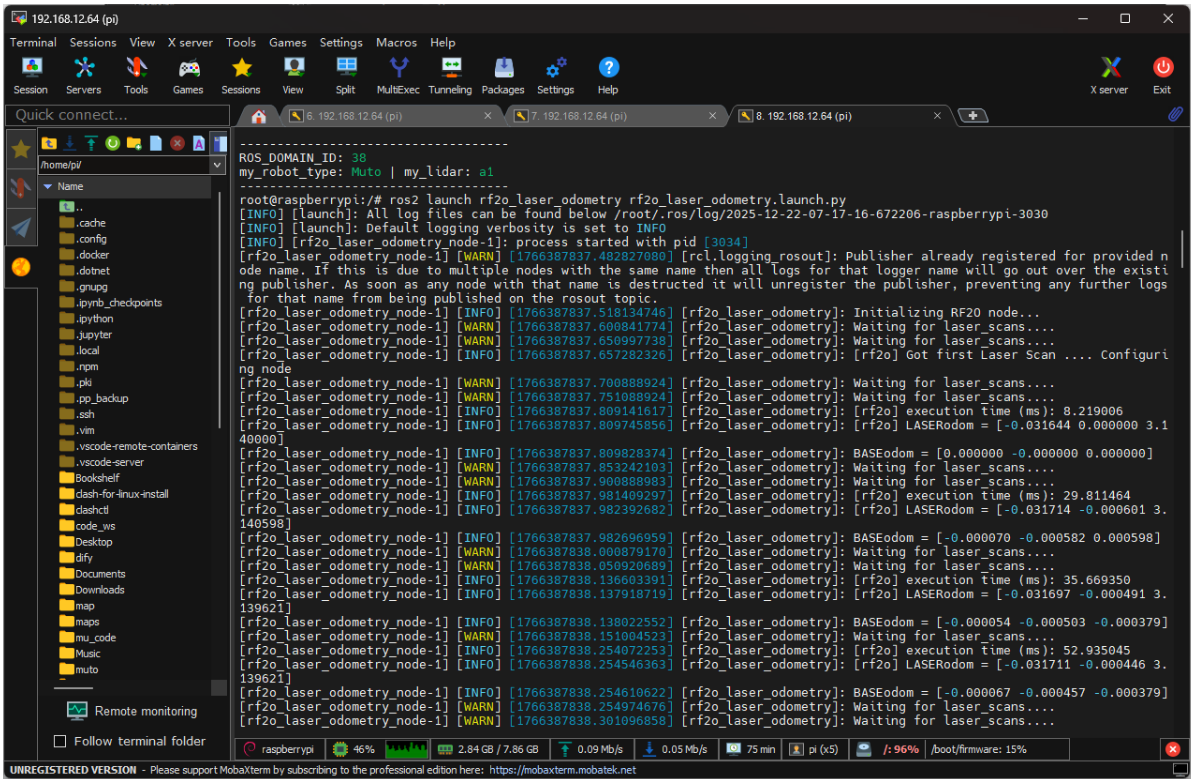
First terminal, start the radar node

```
ros2 launch yahboomcar_nav laser_bringup_launch.py
```



In the second terminal, we need to convert the radar data into odometry data for use by the navigation node.

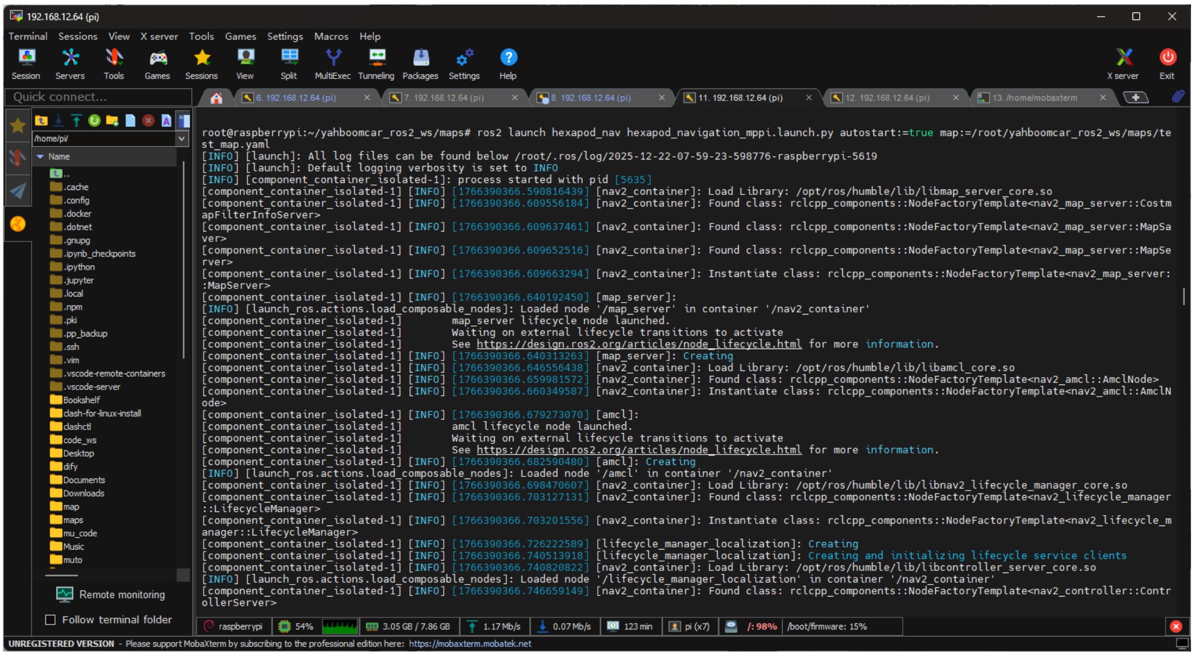
```
ros2 launch rf2o_laser_odometry rf2o_laser_odometry.launch.py
```



Next, we start the navigation node. Here we choose to use the MPPI planner, which performs better in narrow terrain.

```
ros2 launch hexapod_nav hexapod_navigation_mppi.launch.py autostart:=true
map:=/root/yahboomcar_ros2_ws/maps/test_map.yaml
```

Please note that a pre-built map file (.yaml and .pgm, here we are using the map /root/yahboomcar_ros2_ws/maps/test_map) is required.



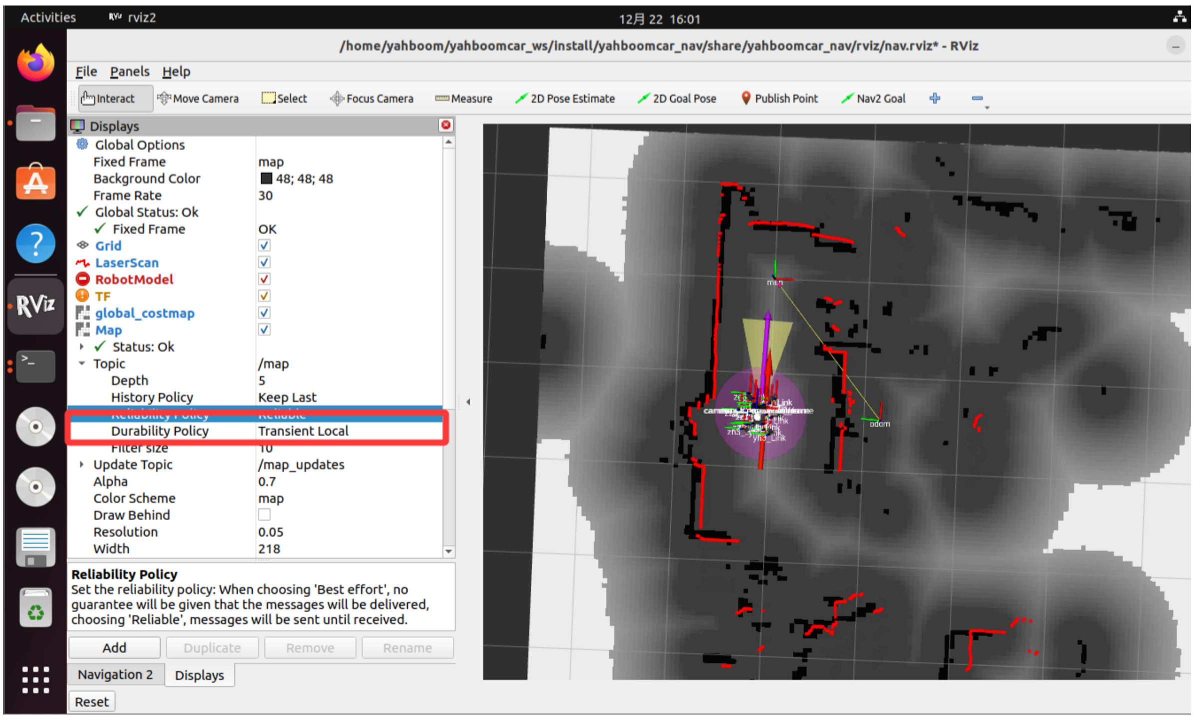
Finally, we can start the Rviz2 node to easily observe the navigation process.

This step is recommended to be performed in a VMware virtual machine!! **

```
ros2 launch yahboomcar_nav display_nav_launch.py
```

It is crucial to change the selected area in the image to Transient Local; otherwise, the map may not load.

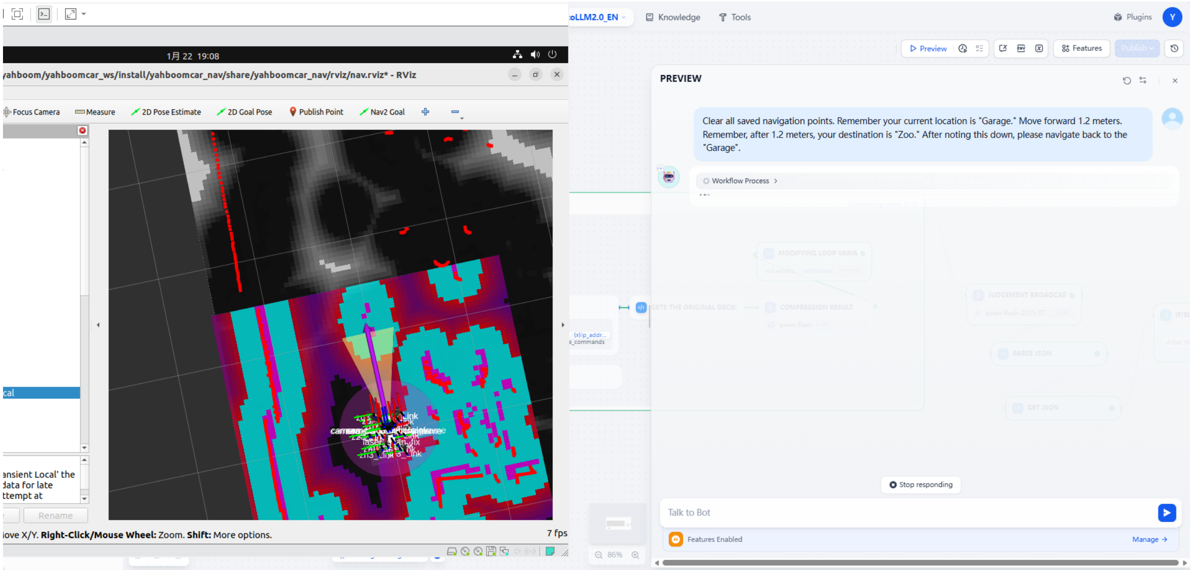
After selecting it, we use the 2D Pose Estimate tool to set the initial position.



After completing the above steps, we can use the large model to control Muto navigation!

We can enter the following commands:

```
Clear all saved navigation points. Remember your current location is "Garage."
Move forward 1.2 meters. Remember, after 1.2 meters, your destination is "Zoo."
After noting this down, please navigate back to the "Garage".
```



As you can see, navigation **may** fail. The large model will analyze the possible reasons for the failure based on the error message.

The error message for this failure is as follows:

```
[component_container_isolated-1] [INFO] [1766392121.987407276]
[controller_server]: Received a goal, begin computing control effort.
............
[component_container_isolated-1] [INFO] [1766392121.994916498]
[controller_server]: Optimizer reset
[component_container_isolated-1] [WARN] [1766392123.016680997]
[BehaviorTreeEngine]: Behavior Tree tick rate 100.00 was exceeded!
[component_container_isolated-1] [INFO] [1766392123.087624461]
[controller_server]: Passing new path to controller.
..............
[component_container_isolated-1] [WARN] [1766392135.611962279]
[BehaviorTreeEngine]: Behavior Tree tick rate 100.00 was exceeded!
[component_container_isolated-1] [WARN] [1766392167.215183496]
[BehaviorTreeEngine]: Behavior Tree tick rate 100.00 was exceeded!
..................
[component_container_isolated-1] [INFO] [1766392167.220835206]
[global_costmap.global_costmap]: Message Filter dropping message: frame 'laser'
at time 1766392166.773 for reason 'the timestamp on the message is earlier than
all the data in the transform cache'
[component_container_isolated-1] [ERROR] [1766392167.349428846]
[controller_server]: Failed to make progress
[component_container_isolated-1] [WARN] [1766392167.349622410]
[controller_server]: [follow_path] [ActionServer] Aborting handle.
................
[component_container_isolated-1] [WARN] [1766392167.375357691] [bt_navigator]:
[navigate_to_pose] [ActionServer] Aborting handle.
[component_container_isolated-1] [ERROR] [1766392167.375855844] [bt_navigator]:
Goal failed
```

This is because different geographical environments require different navigation configuration files.

We can modify the `hexapod_nav_mppi_params.yaml` configuration file to find suitable parameters for navigation based on the actual environment.

Manually Configuring Navigation Points

Besides letting Muto automatically record navigation points, we can also manually set them.

1. Configuration File Location

The navigation point configuration file **does not exist** by default. It will only be automatically generated by the system after you first call the save position function (e.g., `save_current_position`).

According to the code logic, the complete path of the generated file is:

`~\muto-llm-2.0\packages\config\saved_positions.json`

2. How to Manually Add Navigation Points

Since this file has not yet been generated, you can choose one of the following two methods:

Method 1: Let the System Generate Automatically (Recommended)

The simplest method is to have the robot perform a save operation at the current position (through voice commands or code calls). The system will automatically create the directory and file structure. Afterwards, you only need to open the file and modify the coordinates.

Method 2: Manually Create the File

1. Create a folder named `config` in the `~\muto-llm-2.0\packages\` directory.
2. Create a file named `saved_positions.json` inside the `config` folder.
3. Copy the following JSON template content into the file and modify it to your desired coordinate parameters.

File Content Template (`saved_positions.json`):

```
{
  "living_room": {
    "position": {
      "x": 1.5,
      "y": 2.0,
      "z": 0.0
    },
    "orientation": {
      "x": 0.0,
      "y": 0.0,
      "z": 0.0,
      "w": 1.0
    },
    "saved_time": 1705300000.0,
    "saved_time_str": "2024-01-15 12:00:00",
    "frame_id": "map",
    "child_frame_id": "base_link"
```

```json
  },
  "kitchen": {
    "position": {
      "x": -1.0,
      "y": 3.5,
      "z": 0.0
    },
    "orientation": {
      "x": 0.0,
      "y": 0.0,
      "z": 0.707,
      "w": 0.707
    },
    "saved_time": 1705300100.0,
    "saved_time_str": "2024-01-15 12:05:00",
    "frame_id": "map"
  }
}
```

3. Parameter Description

- **Key (e.g., "living_room")**: The name of the navigation point, used in voice commands.
- **position**: The X/Y/Z coordinates of the target point (in meters).
- **orientation**: The orientation of the target point, represented by quaternions (x, y, z, w).
- **frame_id**: Usually fixed as `"map"`, indicating a map-based coordinate system.